

Experiment No.	1
Experiment Title	Working with Python Packages – NumPy, SciPy, Scikit-learn, Matplotlib
Student Name	Isha Venkataraman
Register Number	3122247001023
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To work directly with the core Python libraries used in a machine learning workflow — NumPy, Pandas, SciPy, Scikit-learn and Matplotlib — by applying them to five different real-world datasets (loan approval, iris flowers, diabetes, spam messages, handwritten digits). Each dataset is run through the same basic pipeline: load, explore, clean, select features, split, train and evaluate. For each dataset the type of ML problem and a suitable algorithm are identified rather than given upfront.

2 Problem Statement

The challenge is to take five raw CSV files with different mixes of numeric, categorical, text and pixel data and decide on a sensible target variable for each, identify the type of learning problem it represents and train and evaluate a basic model. All five are classification problems—loan approval, iris species, diabetes diagnosis, spam detection, and digit recognition—but the nature of the features, structured health readings, free text, raw pixel values, means that the preprocessing and feature selection steps are different across the problems, even when the same reusable functions are doing most of the work.

3 Dataset Description

Loan Amount Prediction	Kaggle, Loan Prediction dataset. 614 applications, 12 usable features (income, credit history, education, property area, etc.) after dropping the ID column, target is Loan Status (approved / not approved). Several columns have real missing values (Credit History alone is missing in 50 rows). Split 80/20, stratified.
Iris Dataset	Classic UCI/Kaggle Iris dataset. 150 flower measurements, 4 numeric features (sepal/petal length and width), target is Species (3 classes, 50 each). No missing values, though 3 exact duplicate rows appeared once loaded. Split 80/20, stratified.

Predicting Diabetes	Kaggle, Pima Indians Diabetes dataset. 768 patients, 8 numeric health features, target is Outcome (diabetic / not). No values are technically NaN, but several columns use 0 as a stand-in for a missing reading. Split 80/20, stratified.
Email/SMS Spam Classification	Kaggle/UCI, SMS Spam Collection. 5,572 text messages, labelled ham or spam.
Handwritten Digit Recognition (MNIST)	Kaggle CSV format for MNIST. 60000 images each flattened into 784 pixel columns (28x28); target is the digit label 0–9, fairly evenly distributed across all ten digits. The SVM training was performed on a random sample of 6000 rows to keep the runtime reasonable.

4 Exploratory Data Analysis

For the three plain tabular datasets (Loan, Iris, Diabetes) the same reusable `perform_eda()` function was used to produce the same style of 12-plot grid (missing values, correlation heatmap, histogram, scatter, box plot, bar chart, pie chart, violin plot, density plot, regression plot, strip plot, ECDF) for each. Spam and MNIST, needed some extra processing to make this function make sense to use, as detailed in their own subsections below.

Loan Dataset EDA

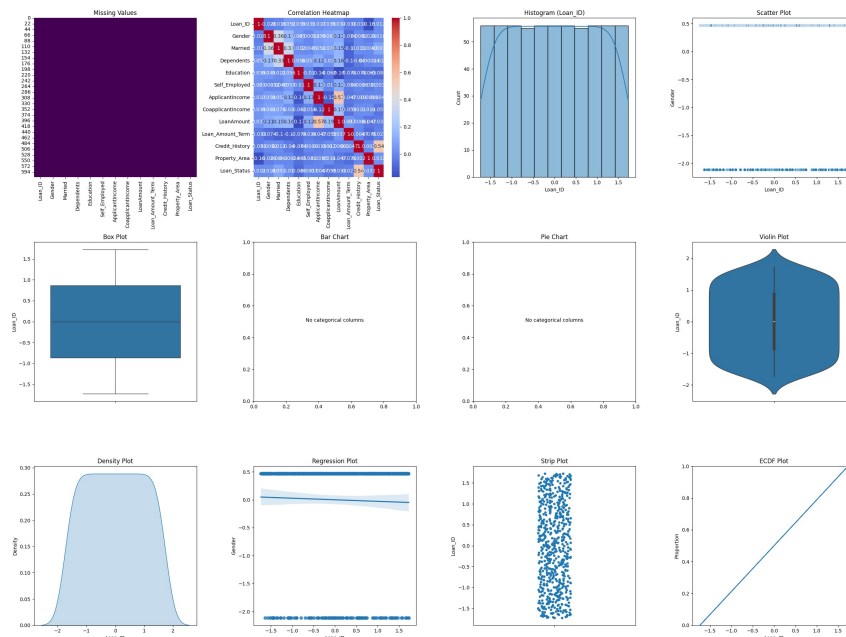


Figure 1: 12-plot EDA grid for the preprocessed Loan dataset.

Inference: The heatmap of missing values is almost empty, because when calling this function, the missing values are already imputed in the preprocessing step (median for numbers, mode for categories). The correlation heatmap is more informative. ApplicantIncome and CoapplicantIncome do not correlate strongly with Loan Status directly, while Credit History, is the feature that is most associated with approval, which is in line with what is commonly reported for this dataset. Here the bar and pie charts are of the encoded Loan ID column as it is the first categorical column in the data. This is a side effect of the function always picking the first column it finds, not the one that, makes the most sense. The plot contains no information. The plot contains no information.

Iris Dataset EDA

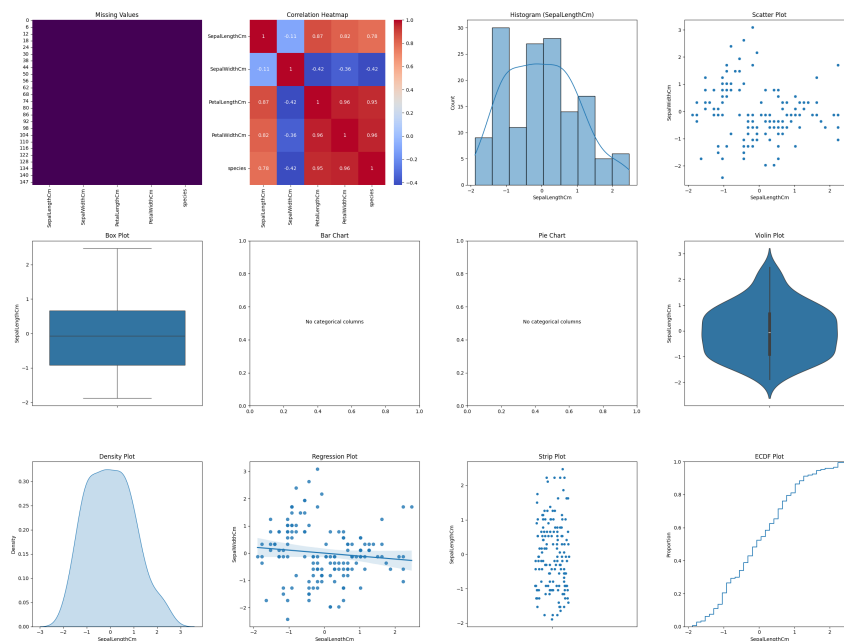


Figure 2: 12-plot EDA grid for the preprocessed Iris dataset.

Inference: Iris is a tidy data set, as is evident throughout this grid. The correlation heatmap shows that petal length and petal width are strongly correlated with each other (around 0.9+), indicating that larger flowers tend to have larger petals in both dimensions simultaneously. Three rows turned out to be exact copies of one another, and were dropped automatically. This took the dataset from 150 rows to 147 – a known quirk of some copies of Iris in CSV format that circulate on the internet. The scatter plot of sepal length vs sepal width is a lot messier than one might expect; the three species are not well separated by sepal measurements alone (something that will become relevant later when we talk about feature selection).

Diabetes Dataset EDA

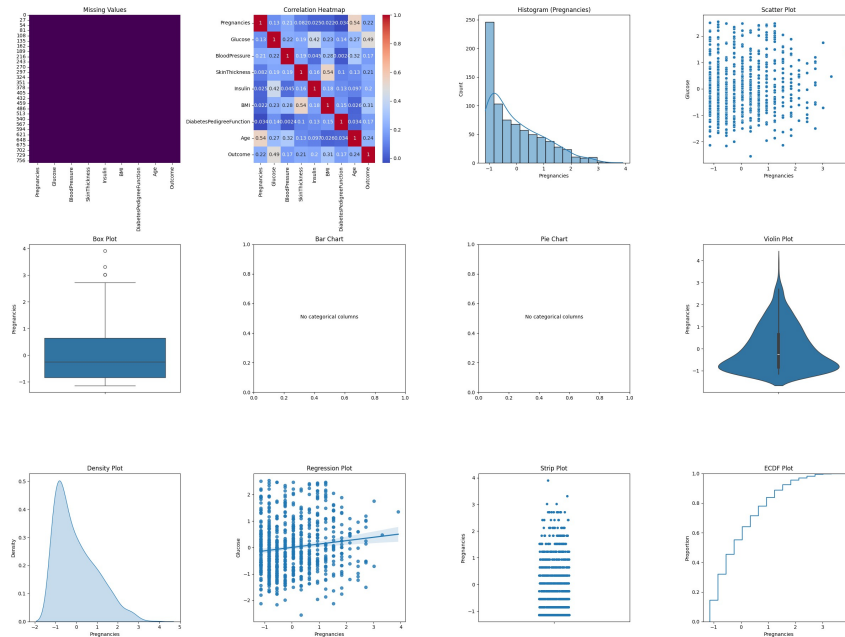


Figure 3: 12-plot EDA grid for the preprocessed Diabetes dataset (after fixing disguised missing values, see Section 6).

Inference: The correlation heatmap is super useful: Glucose is the feature most correlated with Outcome, since high blood sugar is one of the main diagnostic markers for diabetes. The histogram of Pregnancies (the first numeric column) is highly skewed to the right because most patients have had a small number of pregnancies and a few patients have had many. This is also the dataset where the missing-value story is more complicated than the heatmap suggests – the heatmap checks for real NaN values, and by this stage the disguised zero values have already been converted and filled in, so the data looks clean here even though the raw file was not.

Spam Dataset EDA

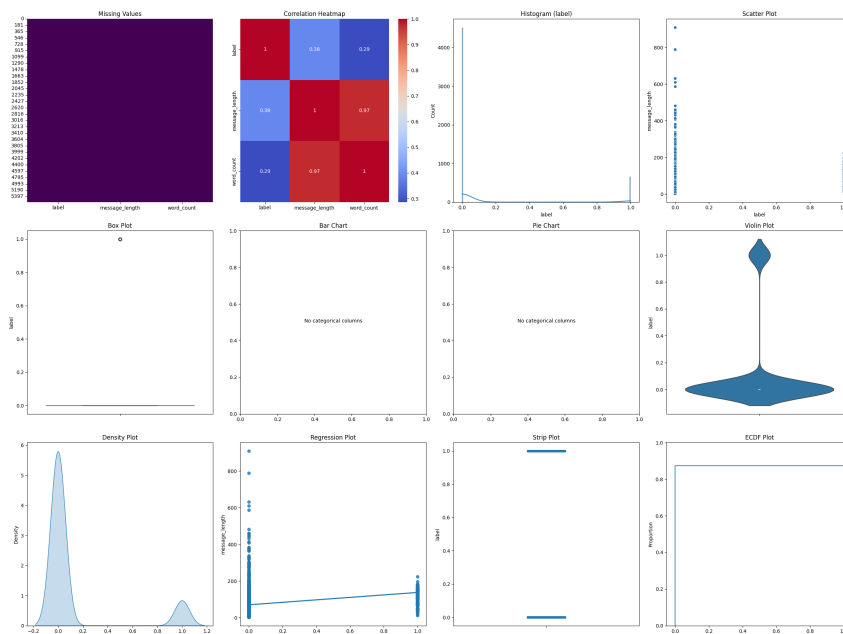


Figure 4: EDA grid run on two derived numeric features (message length, word count) plus the encoded label, since raw text cannot go through the same numeric-focused function directly.

Inference: Since `perform_eda()` expects numeric columns to plot anything meaningful, two simple numeric stand-ins were computed first – message length in character and word count – and passed in instead of raw text. The bar and pie charts of the label confirm what the value counts already show: spam, is a clear minority class, close to 1 in 8 messages. The length-based plots show a long right tail, with most texts short but a noticeable stretch of longer ones; cross-referencing this with the label shows longer messages skew spammy, consistent with spam texts commonly packing in a link, a phone number and some urgency in one message.

MNIST Sample Images and EDA

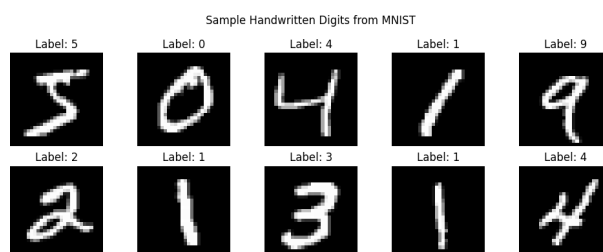


Figure 5: First 10 images from the MNIST CSV, reshaped back into 28x28 and shown with their true labels.

Inference: This is more of a sanity check than a proper EDA plot, to see if the 784 pixel columns reconstruct into images that match the label that they are supposed to

represent. Some are a little blurry to read at a glance at 28 x 28 resolution but that is just a property of how small these images are and not an issue of the loading.

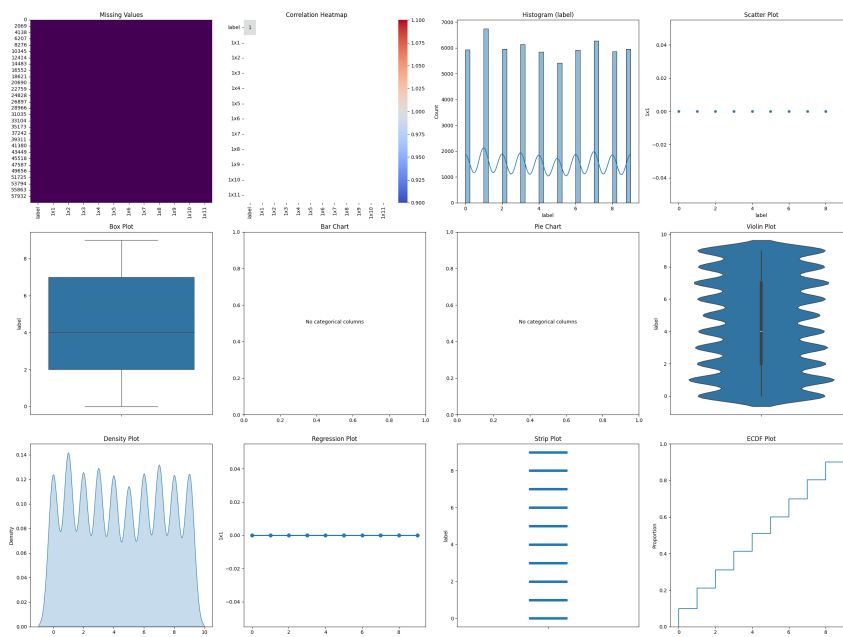


Figure 6: EDA grid run on the label column plus the first 11 raw pixel columns.

Inference: Passing all 784 pixel columns at once to the full EDA function would produce a correlation heatmap that is impossible to read. So, only the label and the first 11 pixel columns were passed in. The bar and pie charts of the label are pretty evenly distributed over all ten digits, which means that accuracy alone is a fairly good way to evaluate the model later on, as no single digit dominates the dataset. The histogram/box/violin plots of the very first pixel column is basically a single spike at 0, as that corner of the image is almost always black no matter what digit is drawn – just the sort of pixel that is later removed by variance thresholding.

5 Data Preprocessing

The heart of the preprocessing for the three tabular datasets is one function, `load_and_preprocess()`, written once and reused for Loan, Iris and Diabetes instead of writing separate cleaning code three times. Given any CSV file, it drops duplicate rows, drops any rows, or columns that are completely empty, handles common missing-value placeholders such as “?”, removes empty strings as real NaN, removes stray whitespace from text columns, converts numeric looking text into real numbers, fills in missing numeric values with the column median and missing categorical values with the column mode, label-encodes any other text columns and finally standardizes every numeric feature column except the last (assumed to be the target).

- **Missing value handling:** Handled automatically by the function above for Loan and Diabetes; no missing values needed handling for Iris. Diabetes required one extra step first – several columns (Glucose, BloodPressure, SkinThickness, Insulin, BMI) use a literal 0 to mean “not recorded” instead of leaving the cell blank, and a value of 0 for, something like blood pressure is not physiologically real. Since

the generic function only fills in actual NaN values, these disguised zeros were converted to NaN first (374 of them in Insulin alone, out of 768 rows) before the file was handed to the shared pipeline.

- **Duplicate rows:** Iris had 3 exact duplicate rows dropped automatically by the generic function, going from 150 down to 147 rows. Loan and Diabetes had no duplicates.
- **Encoding:** Text columns (Gender, Married, Property Area, Species, etc.) were label-encoded into integers wherever they appeared, since none of the models used here accept raw text as a feature directly.
- **Feature scaling:** All numeric feature columns (excluding the assumed target) were standardized with StandardScaler, so features on very different native scales – income in the thousands versus a 0/1 flag – do not dominate the models purely because of their units.
- **Train-test split:** An 80/20 stratified split, was used for every one of the five datasets, keeping the class balance roughly consistent on both sides.

Spam and MNIST were not passed through this shared function, by design rather than error – `load_and_preprocess()` is designed around a table of numbers and categories, and text messages or raw pixel grids do not fit that shape. Spam messages were cleaned separately (dropping duplicate/empty messages, stripping whitespace, label-encoding ham/spam) and then converted into numeric features using TF-IDF. Chi-Square (SelectKBest) was then applied on top of the resulting TF-IDF vectors to pick out the terms most associated with the spam/ham label. TF-IDF here is the vectorization step, Chi-Square is the actual feature selection step. MNIST pixels are already numeric, so nothing needed encoding; instead of scaling all 784 columns blindly, a variance threshold was used first to drop pixels that barely change across images (mostly blank corner pixels), before scaling and feeding the rest into an SVM.

6 Machine Learning Algorithm

Five different algorithms were used across the five datasets, matched to what each kind of data needed.

Logistic Regression (Loan, Diabetes)

Fits a linear boundary between classes by modelling the log-odds, of the positive class as a linear function of the input features, then squashing that through the sigmoid function to get a probability:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(w^\top x + b)}}$$

Advantages: fast, easy to interpret (the sign and size of each coefficient is informative), works well when classes are roughly linearly separable. **Limitations:** struggles if the true decision boundary is genuinely non-linear, and can be misleading under strong class imbalance if only accuracy is examined.

K-Nearest Neighbors (Iris)

It assigns a new point to a class based on the k nearest neighbors in the training data, and votes between those neighbors. There is no real "training" other than storing the data. **Advantages:** no assumption about the shape of the data. Works well when the classes are well separated like in Iris. **Limitations:** gets slower with larger datasets since each prediction requires a new distance search; needs features to be on a similar scale to work properly.

Multinomial Naive Bayes (Spam)

A probabilistic classifier that assumes features (here, TF-IDF word scores) are conditionally independent given the class, multiplying together per-class, per-word probabilities to decide which class is more likely. **Advantages:** very fast, a strong baseline for text specifically, since word-frequency-style features, are exactly what it was designed around. **Limitations:** the independence assumption does not really hold (word choice is not independent), but it tends to work well anyway for text classification.

Support Vector Machine with RBF kernel (MNIST)

Finds the boundary that maximizes the margin between classes, with the RBF kernel allowing that boundary to be non-linear by implicitly mapping the data into a higher-dimensional space. **Advantages:** handles non-linear boundaries well, a solid classic choice, for image data. **Limitations:** training time grows quickly with the number of rows, which is why only a 6,000-row sample of the full 60,000-row MNIST file was used here.

7 Experimental Setup

Python Version	3.12
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU, no GPU used

8 Hyperparameter Selection

No formal grid search was performed in this experiment, since the focus was getting the overall workflow right across five datasets rather than maximizing accuracy on any one model. The choices below are the defaults or simple, commonly, used values for each algorithm.

Model	Parameters Used
Logistic Regression (Loan, Diabetes)	max_iter=1000, all other defaults
K-Nearest Neighbors (Iris)	n_neighbors=5
Multinomial Naive Bayes (Spam)	defaults (alpha=1.0)
SVM, RBF kernel (MNIST)	kernel="rbf", gamma="scale"

9 Results

Dataset (Model)	Accuracy	Precision	Recall	F1
Loan (Logistic Regression)	0.8618	0.84*	0.99*	0.91*
Iris (KNN, k=5)	0.9667	0.97	0.97	0.97
Diabetes (Logistic Regression)	0.7013	0.58*	0.54*	0.56*
Spam (Multinomial NB)	0.9632	0.93*	0.77*	0.84*
MNIST sample (SVM, RBF)	0.9317	0.93	0.93	0.93

*figures shown are for the minority/positive class specifically (loan approval, diabetic, spam), since these three datasets are imbalanced and majority-class numbers alone would make the models look better than they really are on the class that actually matters.

ROC-AUC was not computed for every dataset here, since this experiment's focus is the overall workflow across five datasets, rather than a deep evaluation of a single model, that kind of full metric suite (ROC-AUC, cross-validation, hyperparameter tuning) is covered in depth for a single dataset in Experiment 2.

10 Result Visualization

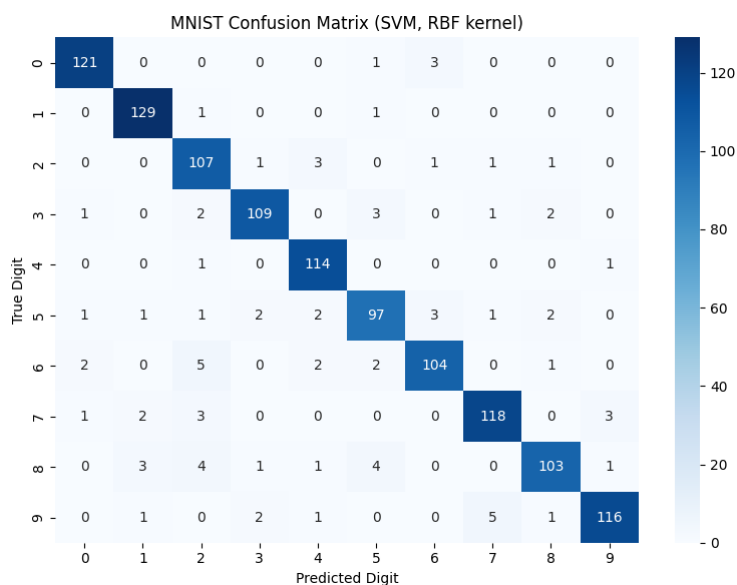


Figure 7: Confusion matrix for the MNIST digit classifier (SVM, RBF kernel, 6,000-row sample, 20% held out for testing).

Inference: Most of the weight in this matrix sits on the diagonal, matching the 93% accuracy reported above. Digit 2 has the lowest precision (0.86) of the ten, meaning a fair number of other digits get misread as a 2, and digit 5 has the lowest recall (0.88), meaning actual 5s are the ones most often missed. This is a fairly normal pattern for handwritten digit recognition – some digit shapes genuinely overlap more than others when handwritten quickly or messily, so a small amount of confusion between visually similar digits is expected even from a strong classifier.

11 Discussion

Putting a generic `load_and_preprocess()` function and a generic `perform_eda()` function through five genuinely different datasets was a useful way to see where "generic" actually holds up and where it starts to strain. For the three plain tabular datasets it worked well and saved a fair amount of repeated code, but Diabetes needed one dataset-specific fix before the generic function could do the right thing (the disguised zero values), a reminder that a reusable function still requires understanding what is actually in the data rather than trusting it blindly on any CSV.

The EDA function's habit of always plotting the first numeric or categorical column also turned out to be worth noticing. Sometimes that first column happened to be the interesting one (Pregnancies for Diabetes, Glucose showing up in the correlation heatmap), and sometimes it very much was not (the encoded Loan ID for Loan, which is essentially a random number and carries no information). That is the trade-off of a fully generic function – consistency and saved time versus giving up the function automatically knowing which column is worth looking at.

Text and images required separate pipelines, since applying generic preprocessing to TF-IDF vectors or raw pixel grids would be ineffective. Feature selection also varied across datasets: chi-square for Loan, ANOVA F-test for Iris and Diabetes, TF-IDF + chi-square for Spam, and variance thresholding for MNIST. This was probably the single biggest practical takeaway, since there is no universal feature selection method; it depends on the data.

Several datasets were noticeably imbalanced (Loan, Diabetes, Spam), so accuracy alone hides weaker performance on minority classes. Diabetes, for example, achieved only 0.54 recall for the diabetic class despite reasonable overall accuracy. Improving these models would require focusing on the minority class rather than relying on overall accuracy.

12 Conclusion

This experiment provided hands-on experience with the complete machine learning workflow across five diverse datasets. Iris achieved the highest accuracy (96.7%), reflecting its well-separated classes, while Loan and Spam showed good accuracy but were influenced by class imbalance. Diabetes proved the most challenging tabular dataset, whereas MNIST achieved a strong 93% accuracy with an SVM despite using a reduced sample and feature set. Overall, the experiment demonstrated that a standard preprocessing and EDA pipeline works well for many tabular datasets, but text and image data require specialized approaches, and feature selection methods must be chosen based on the dataset.

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Supervised, Multi-class Classification	ANOVA F-test (SelectKBest)	K-Nearest Neighbors
Loan Amount Prediction	Supervised, Binary Classification	Chi-Square test (SelectKBest)	Logistic Regression
Predicting Diabetes	Supervised, Binary Classification	ANOVA F-test (SelectKBest)	Logistic Regression

Email/SMS Spam Classification	Supervised, Binary Classification	TF-IDF + Chi-Square test (SelectKBest)	Multinomial Naive Bayes
Handwritten Character Recognition / MNIST	Supervised, Multi-class Classification	Variance Threshold	SVM (RBF kernel)

Table 2: Summary of ML task type, feature selection technique and suitable algorithm for each dataset.

13 References

1. M. Hopkins et al., "Spambase / SMS Spam Collection," UCI Machine Learning Repository. [Online]. Available: <https://archive.ics.uci.edu/>
2. R. A. Fisher, "The Use of Multiple Measurements in Taxonomic Problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
3. Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits." [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
4. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
5. Kaggle, "Loan Prediction Problem Dataset" and "Pima Indians Diabetes Database." [Online]. Available: <https://www.kaggle.com/datasets>